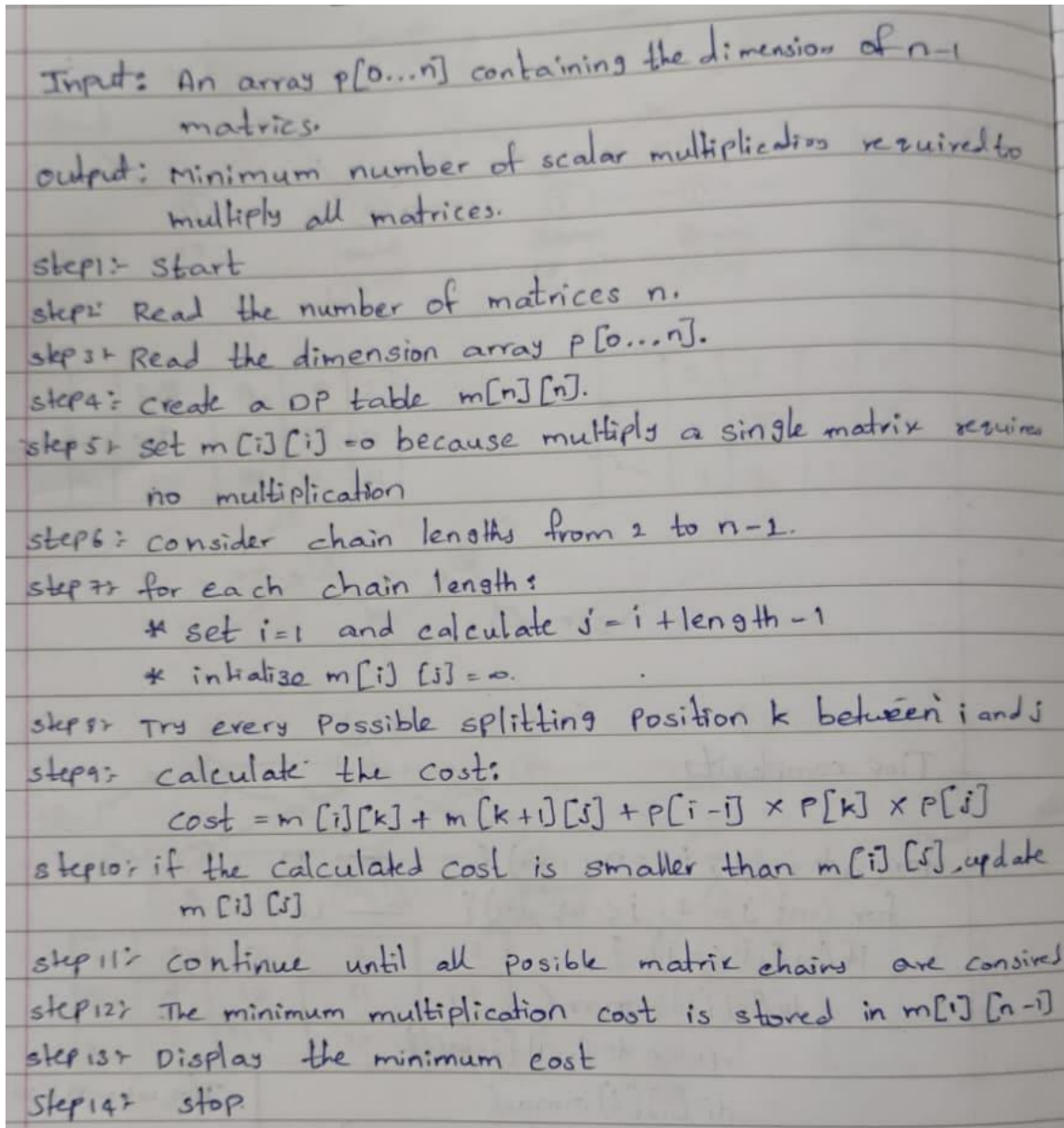


PRACTICAL - 6

AIM : Implementation of chain matrix multiplication using dynamic programming.

MATRIX CHAIN MULTIPLICATION

ALGORITHM :



Inputs: An array $p[0...n]$ containing the dimension of $n-1$ matrices.
Output: Minimum number of scalar multiplications required to multiply all matrices.

step1:- start
step2:- Read the number of matrices n .
step3:- Read the dimension array $p[0...n]$.
step4:- create a DP table $m[n][n]$.
step5:- set $m[i][i] = 0$ because multiply a single matrix requires no multiplication
step6:- consider chain lengths from 2 to $n-1$.
step7:- for each chain length:
 * set $i=1$ and calculate $j=i+length-1$
 * initialize $m[i][j] = \infty$.
step8:- Try every possible splitting position k between i and j
step9:- calculate the cost:
 $cost = m[i][k] + m[k+1][j] + p[i-1] \times p[k] \times p[j]$
step10:- if the calculated cost is smaller than $m[i][j]$, update $m[i][j]$
step11:- continue until all possible matrix chains are considered
step12:- The minimum multiplication cost is stored in $m[1][n-1]$
step13:- Display the minimum cost
step14:- stop

CODE:

```
#include <iostream>
#include <climits>
using namespace std;
```

```
class MatrixChain {
private:
    int p[20];
    int n;
    int C[20][20];

public:

    void getInput() {
        cout << "Enter number of matrices: ";
        cin >> n;

        cout << "Enter dimensions: ";
        for (int i = 0; i <= n; i++) {
            cin >> p[i];
        }
    }

    void solve() {

        // Cost of multiplying one matrix is 0
        for (int i = 1; i <= n; i++) {
            C[i][i] = 0;
        }

        // length is the chain length
        for (int length = 2; length <= n; length++) {

            for (int i = 1; i <= n - length + 1; i++) {

                int j = i + length - 1;
                M[i][j] = INT_MAX;

                for (int k = i; k < j; k++) {

                    int cost = C[i][k] + C[k + 1][j] + p[i - 1] * p[k] * p[j];

                    if (cost < C[i][j]) {
                        C[i][j] = cost;
                    }
                }
            }
        }
    }
}
```

```
void printTable() {  
  
    cout << "\nDP Table:\n";  
  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= n; j++) {  
  
            if (j < i)  
                cout << "-\t";  
            else  
                cout << C[i][j] << "\t";  
        }  
  
        cout << endl;  
    }  
}  
  
void showResult() {  
    cout << "\nMinimum number of scalar multiplications = "  
        << C[1][n] << endl;  
}  
};  
  
int main() {  
  
    MatrixChain m;  
    m.getInput();  
  
    m.solve();  
    m.printTable();  
    m.showResult();  
  
    return 0;  
}
```



OUTPUT:

```
Enter number of matrices: 4
Enter dimensions: 4 5 6 3 2

DP Table:
0      120      150      136
-      0        90      96
-      -        0      36
-      -        -      0

Minimum number of scalar multiplications = 136
```

TIME COMPLEXITY:

best case :

Time complexity

for length = 2 to n-1 — runs n times
 for i = 1 to n-length — (n)
 for k = i to j-1 — (n)
 calculate cost
 $n \times n \times n = O(n^3)$

average case :

Time complexity

for length = 2 to n-1 — runs n times
 for i = 1 to n-length — (n)
 for k = i to j-1 — (n)
 calculate cost
 $n \times n \times n = O(n^3)$



worst case :

